

Exercícios – Objetivo 105.1

Parte 1

1. Quando o Bash é invocado como shell com login, ele pode executar vários arquivos de inicialização:

```
/etc/profile
~/.profile
~/.bash_profile
~/.bash_login
```

O *profile* global (/etc/profile) é executado primeiro, seguido pelo(s) *profile(s)* do usuário. Edite os arquivos de *profile* que ficam no seu diretório home, adicionando ao final uma linha como (use uma mensagem distinta para cada arquivo):

```
echo "executando .profile"
```

Caso algum arquivo não exista, ele deverá ser criado. Certifique-se de que os arquivos tenham permissão de leitura para o dono.

Em seguida, invoque o Bash como shell com login, e identifique se todos os arquivos de *profile* são executados, e em que sequência. Caso nem todos os arquivos sejam executados, renomeie os que foram executados (por exemplo, `mv ~/.profile ~/.profile.old`) e repita a invocação, até que todos os arquivos tenham sido executados pelo menos uma vez.

Ao final, descreva como o Bash trata os arquivos de *profile* do usuário. Se você quiser e tiver privilégios para isto, pode incluir /etc/profile no experimento.

ATENÇÃO: ao final do exercício, certifique-se de restaurar os arquivos que já existiam antes no sistema.

2. Quando uma variável é exportada usando `export`, ela se torna uma variável do shell. Isso pode ser verificado na prática:

```
$ var1=bagre
$ var2=frito
$ export var1
$ bash -c 'echo $var1 $var2'
```

- (a) Qual a diferença na saída se, no último comando, as aspas simples forem substituídas por aspas duplas? Por que isso acontece?
- (b) O que acontece se uma variável já exportada for modificada? Essa mudança é refletida automaticamente em um sub-shell que herde essa variável?
- (c) O que acontece com uma variável exportada se ela for modificada em um sub-shell?

```
$ var1=bagre    # estamos no shell principal
$ export var1
$ bash
$ var1=baiacu   # estamos no sub-shell
$ kill -STOP $$ # suspende o sub-shell
$ echo $var1    # voltamos ao shell principal
$ fg           # retorna ao sub-shell
```

(d) Atribua o conteúdo de uma variável a uma outra variável, e imprima a nova variável:

```
$ var3="ensopado de $var1"
$ echo $var3
```

O que acontece com var3 se var1 for modificada após a atribuição? O uso de export faz alguma diferença?

3. Os nomes de variáveis do shell são sensíveis a caso? Verifique isso na prática.
4. O comando unset remove a definição de uma variável. Qual o efeito de unset sobre uma variável de ambiente (i.e., uma variável que tenha sido exportada com export)?
5. Como reverter a exportação de uma variável de ambiente sem remover a variável de shell, como no exemplo abaixo?

```
$ zz1=abacaxi ; export zz1
$ env | grep zz
zz1=abacaxi
$ ???      # que comando(s) colocar aqui?
$ env | grep zz
$ echo $zz1
abacaxi
```

6. Crie três arquivos chamados baleia, azul e 'baleia azul', com conteúdos distintos, e um diretório d1. A seguir, execute os comandos:

```
$ arq=azul
$ cp $arq d1
```

- (a) O que fazem esses comandos?
- (b) O que acontece se você mudar o primeiro comando para arq='baleia azul'?

7. "Bananada" é um doce de banana, assim como "goiabada" é um doce de goiaba. No shell, atribua à variável local fruta o nome de uma fruta, e à variável local doce o valor de fruta acrescido do sufixo "da".

IMPORTANTE: você deve usar a variável fruta, e não escrever o nome da fruta desejada, como no exemplo abaixo:

```
$ fruta=banana
$ ???      # que comando(s) colocar aqui?
$ echo $doce
bananada
```

8. Dois comandos que podem ser usados para listar as variáveis de ambiente do shell são env e printenv. Qual a diferença entre as saídas desses comandos?

DICA: redirecione as saídas dos comandos para arquivos e compare-as com diff.

9. Existem diversas variáveis de ambiente cujos valores são atribuídos pelo sistema. Dois exemplos são USER, que contém o login do usuário, e PWD, que armazena o diretório atual.

- (a) Mostre o conteúdo atual dessas variáveis.
- (b) Modifique o conteúdo dessas variáveis, e veja se isso produz algum efeito. Você pode usar o comando whoami para consultar o que o SO assume como login do usuário.

10. Execute os seguintes comandos:

```
$ mkdir d1
$ echo echo root >d1/whoami
$ chmod 755 d1/whoami
$ PATH=.:${PATH}
$ whoami
$ cd d1
$ whoami
```

Com base no que você observou, explique por que não se deve colocar o diretório corrente no PATH.

Parte 2

11. Na maioria das distribuições Linux, existe um alias de `ls` para que seja mostrada a saída colorida.

```
$ alias ls
alias ls='ls --color=auto'
```

- (a) Verifique se esse alias está definido no seu shell. Se não estiver, defina-o como mostrado acima.
- (b) Defina um alias `lsi` que invoque `ls` com as opções `-li`. Teste o alias, verificando se ele mostra a saída colorida.
- (c) Remova o alias definido originalmente para `ls`. Verifique se agora os comandos `ls` e `lsi` mostram saída colorida.

12. Defina um alias `E` e uma função no shell que tenham o mesmo nome mas façam coisas distintas:

```
$ alias xpto="echo alias"
$ function xpto {
> echo funcao
> }
```

O que acontece quando você invoca `xpto`? Existe alguma forma de invocar a outra definição de `xpto`?

13. Defina com uma única linha de comando uma função `peixe` no shell que apenas imprima o nome de um peixe qualquer (manjuba, se faltar criatividade).

- (a) Use `declare -F` para confirmar que `peixe` aparece na lista de funções do shell, e execute a função para confirmar que ela funciona corretamente.
- (b) Use `declare -f` para exibir a definição da função.
- (c) Use `bash -c peixe` para executar essa função em um sub-shell. O que acontece?
- (d) Exporte a função e repita o passo anterior.
- (e) Crie uma variável de shell `peixe`, atribuindo-lhe qualquer conteúdo. Essa operação funciona corretamente?
- (f) Verifique se `peixe` aparece na saída dos comandos `set` e/ou `env`. Que comando(s) mostra(m) a função `peixe`? Que comando(s) mostra(m) a variável `peixe`?
- (g) Remova a definição da função, e tente executá-la em um sub-shell com `/bin/bash -c peixe`.
- (h) Use `peixe` para tentar executar a função no shell atual. A saída é exatamente idêntica à da tentativa anterior? A que podem ser atribuídas as diferenças?

14. Defina uma função no shell que imprima o número de parâmetros recebidos e quais são esses parâmetros. Exemplo de uso:

```
$ param a b
A funcao recebeu 2 parametros.
Os parametros sao: a b
```

```
$ param a b c d
A funcao recebeu 4 parametros.
Os parametros sao: a b c d
```

15. O que acontece caso a função do exercício anterior receba um ou mais parâmetros contendo espaços? Um parâmetro que contenha espaço será contado uma vez ou a contagem vai depender do seu número de palavras? Quando os parâmetros são mostrados, é possível diferenciar parâmetros com e sem espaços?
16. Defina um alias ou função no shell para buscar em um arquivo linhas que contenham expressões **entre** aspas. Exemplo de uso:

```
$ cat arq
anta
"baleia"
"inicio
formiga
""nao
"tigre" siberiano
final"
```

```
$ aspas arq
"baleia"
"tigre" siberiano
```

17. Defina um alias ou função no shell que procure os arquivos com nome Makefile **OU** com extensão .c ou .h sob o diretório passado como parâmetro. Exemplo de uso:

```
$ findc ~/proj
~/proj/foo/Makefile
~/proj/foo/foo.c
~/proj/foo/lib/x86_32.h
~/proj/foo/lib/x86_64.h
~/proj/bar/Makefile
~/proj/bar/utils.h
~/proj/bar/lista.h
~/proj/bar/main.c
```